

FRED TALKS

7th July 2026

CONTENTS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

HAN, NOT SOLO · 1766 WORDS

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 1426 WORDS

The future of enterprise software

AARON LEVIE · 1851 WORDS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

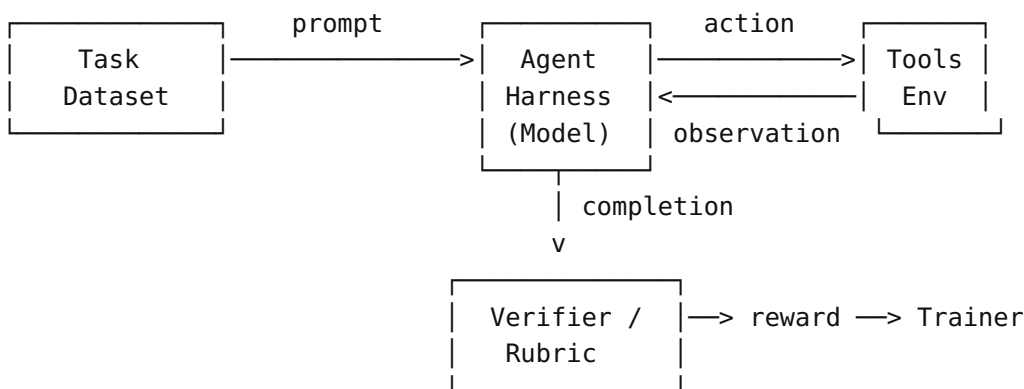
HAN, NOT SOLO · 22 MAR 2026 · [SOURCE](#)

Model architecture gets all the attention. Post-training recipes follow close behind. The training environment — what the model actually practices on, how its work gets judged, what tools it can use — barely enters the conversation. That’s the part that actually determines what the agent can learn to do.

A model trained only on single-turn Q&A will struggle the moment you ask it to maintain state across a 50-step enterprise workflow. A model trained with a poorly designed reward function will learn to game the metric, not solve the problem. The environment isn’t a detail. It’s half the system.

The Canonical Loop

An RL environment for an LLM agent bundles three things: a dataset of task inputs, a harness for the model, and a reward function to score outputs. The training loop looks like this:



Formally, a complete RL environment is a tuple:

Task distribution, harness, verifier, state management, configuration. Let’s go through each.

The task distribution is the set of problems the agent trains on. Not all tasks are equal, and not just in difficulty. They vary structurally in ways that demand different capabilities:

Task Type	What the Agent Must Do	Example Systems
Single-turn Q&A	One prompt → one response, check answer	Math benchmarks, SimpleQA
Multi-hop search	Chain searches, synthesize sources	BrowseComp, WebWalkerQA
Open-ended research	No single correct answer; report quality matters	ADR-Bench, ResearchRubrics
Agentic tool-use	Call tools correctly in sequence	tau-bench, function-calling benchmarks
Stateful enterprise	Modify persistent DB state, work within access controls	EnterpriseOps-Gym
Code execution	Write code, run it, check outputs	SWE-Bench, LiveCodeBench

Training only on tasks with ground-truth answers produces an agent that’s never learned to reason under ambiguity. Training only in clean, deterministic environments produces an agent that falls apart in production. The task distribution is a design decision with direct consequences.

Task synthesis is increasingly a first-class problem. With real-world research tasks, you rarely have a large labeled dataset. Strategies that have emerged:

- **Back translation:** Start from a desired output, reconstruct the query that would produce it
- **Graph-based synthesis:** Build a knowledge graph, generate multi-hop queries over it
- **Automated environment generation:** Use LLM coding agents to write new environment code. AutoEnv reports ~\$4/env average cost.
- **Curriculum construction:** Order tasks by difficulty and increase complexity during training

The cheapest-to-collect tasks are single-turn with verifiable answers. The most valuable tasks for long-horizon behavior are expensive to construct. This tension drives most environment design decisions.

The harness is the scaffolding that mediates between the model and the environment. It controls how the model interacts, not what it knows.

```
H = {
    rollout_protocol, # SingleTurn | MultiTurn | Agentic
    tools,            # Available tools in this episode
    system_prompt,    # Instructions for the agent
    context_manager,  # How to handle context overflow
```

```
    turn_limit,          # Max interactions per episode
    sandbox,             # Code execution sandbox
    state                # Persistent state across turns
}
```

Rollout protocols range from trivial to complex:

Harness Type	Description	When to Use
Single-Turn	One prompt, one response	Math, factual QA
Multi-Turn	Back-and-forth dialogue	Games, structured tasks
Tool-Use	Model calls tools, receives results	Agent benchmarks
Stateful Tool-Use	Tools modify persistent state	Enterprise workflows, SWE-Bench
Agentic ReAct	Full Thought → Action → Observation loop	Deep research, complex workflows

Tools span a wide taxonomy:

Category	Tools	Deterministic?	Stateful?
Information retrieval	web_search, scholar_search	No (live web)	No
Content extraction	jina_reader, visit, web_scrape	No	No
Code execution	python_interpreter, shell, sandbox	Yes (given same code)	Yes
File operations	file_read, file_write	Yes	Yes
Browser automation	playwright, link_click	No	Yes
Task management	todo, section_write	Yes	Yes

The mix of deterministic/non-deterministic and stateful/stateless tools has real implications for reproducibility and reward assignment. Non-deterministic tools mean two runs of the same trajectory can produce different outcomes — which complicates both debugging and verifier design.

Context management is where most teams underinvest, especially for long-horizon tasks. A 600-turn research episode blows past any practical context window. Strategies used in production:

Strategy	Description	Trade-off
Recency-based retention	Keep N most recent turns	Simple, but loses early context
Markovian reconstruction		Principled, expensive

	Reconstruct state from scratch each turn	
Reference-preserving summarization	Summarize old context, keep citations	Preserves verifiability
Reference-preserving folding	Compress context without losing references	Best for research tasks

An agent doing multi-hour research needs to remember why it started searching in a particular direction twelve tool calls ago. Dropping that context causes repeated work and lost threads.

The verifier maps a completion to a reward:

In Atari, the score is unambiguous. In deep research, what counts as a good answer is not. The verifier is where this gets hard.

Type	Reward Signal	When to Use
Exact match	Binary (0/1)	Ground truth available
Code execution	Binary or partial	Output can be tested programmatically
LLM-as-judge	Continuous [0,1]	Open-ended quality, no other option
Checklist-style	Continuous	Multi-criteria research tasks
Evolving rubric (RLER)	Continuous	Resistant to reward hacking
Process reward model (PRM)	Per-step continuous	Long-horizon credit assignment
Pairwise comparison	Relative rank	Relative quality matters more than absolute
Multi-criteria composite	Weighted sum	Multiple quality dimensions

A few principles that actually matter in practice:

Verifiable beats judgeable. Programmatic checks — code execution, string match — are faster, cheaper, and more consistent than LLM-as-judge. Use LLM-as-judge when there’s no other option, not as the default.

Reward granularity is a separate decision from reward type. You can score at the trajectory level (did the final output pass?), turn level (was each tool invocation useful?), or per-step with process rewards. Turn-level supervision, as Nanbeige4.1 does across up to 600 tool calls, enables finer credit assignment — the model can learn that the problem was a bad search query in turn 23, not that the entire episode failed.

Static rubrics get gamed. Models learn to write answers that score well on your rubric rather than solving the problem. DR Tulu’s RLER (Rubric-Level Evolving Reward) co-evolves the rubric with the policy during training. Harder to exploit a moving target.

Noise injection is underrated. Step-DeepResearch deliberately injects 5–10% tool errors during training. The resulting model handles flaky APIs and unexpected failures in production significantly better.

State management determines whether the environment is stateless or persistent. Most academic benchmarks are stateless — each episode starts fresh. Enterprise environments are not. EnterpriseOps-Gym maintains 164 database tables and 512 tools across episodes. Actions in one task affect the state seen by subsequent tasks. That’s a fundamentally different problem for agents to solve.

Configuration covers turn limits, context budgets, sampling temperature, and curriculum scheduling. These are not afterthoughts. A turn limit of 5 vs. 600 changes what skills the agent can develop. AgentScaler uses a two-phase curriculum — fundamental capabilities first, then domain-specific tasks — and the ordering matters. Step-DeepResearch progressively scales context windows from 32K to 128K during mid-training.

The most consequential architectural question: where does the model sit relative to the environment?

Option A: Model outside the environment (decoupled). Model is served via API; the environment calls it at each step. Clean separation. Easy to swap models.

Option B: Model inside the environment (co-located). Model and environment share the same training loop. Lower latency, tighter integration, harder to reuse.

Option C: Split architecture. Trainer, model inference server, and environment are three separate processes communicating via API. This is where the field is landing.

How real systems implement this:

System	Topology	Notes
Prime Intellect verifiers	Option C (Split)	Env is a standalone Python package distributed via Environments Hub
Tongyi DeepResearch	Option B (Co-located)	Tools, context manager, verifiers inside the training pipeline Single-agent ReAct loop embedded in training

Step-DeepResearch	Option B (Co-located)	
MiroThinker	Option C (Split)	Tool servers and sandbox run independently from model
Tinker API	Option A (Decoupled, cloud)	Model stays remote; researcher sends <code>forward_backward + sample</code> calls via API
AutoEnv	Option A (Decoupled)	CoreEnv/ObsEnv/SkinEnv abstraction layers
EnterpriseOps-Gym	Option A (Decoupled)	Containerized sandbox accessible via any model API

The trade-offs:

Dimension	Model Outside (A/C)	Model Inside (B)
Flexibility	Swap models easily; env is reusable	Tighter integration
Scalability	Scale inference and training independently	Must scale everything together
Portability	Env packages are shareable	Env tied to training framework
Latency	Network overhead per tool call	No network overhead
RL compatibility	Works with any RL trainer	Usually tied to one trainer

The field has converged on Option C for production training. Prime Intellect’s architecture — environments as standalone installable packages that communicate with models via OpenAI-compatible API endpoints — is becoming the standard. The payoff: environments are publishable, trainer-agnostic, and inference and environment execution can be parallelized across nodes.

Tinker pushes this further by making even the training compute remote. The researcher controls the algorithm and never touches model weights. The environment’s job is purely generating experience.

Practical Decision Framework

When building an RL environment for an LLM agent:

Do you have ground truth answers?

- Yes → Exact-match or code-execution verifier
- No → LLM-as-judge, checklist, or pairwise comparison

How many tool calls per episode?

- < 5 → Single-turn or simple tool-use env, no context management needed
- 5–50 → Multi-turn with basic context management
- 50–600 → Full agentic env with reference-preserving context management

Where should the model live?

- Experimenting across many environments → Model outside (Option A/C), use Prime Intellect Hub
- Tight RL training loop → Model co-located (Option B)
- No GPU access → Tinker API

What reward granularity?

- Simple tasks → Outcome-level
- Long-horizon tasks → Turn-level (Nanbeige4.1) or process rewards (PRIME)
- Open-ended research → Evolving rubrics (RLER) or checklist-style (Step-DR)

How to scale environments?

- Manual curation → High quality, expensive, this is where you start
- AutoEnv → Automated generation at ~\$4/env
- AgentScaler → Systematic scaling of heterogeneous simulated environments

Three things to pay attention to.

Environment diversity matters as much as environment quality. AgentScaler's key finding is that heterogeneity of environments drives capability breadth in ways that simply adding more data from the same distribution cannot. You need more kinds of environments, not just more environments.

Automated environment generation is viable. At \$4 per generated environment, cost is no longer the bottleneck. The bottleneck is verifier quality — auto-generated environments with weak reward functions will teach the wrong behaviors at scale. ([AutoEnv](#))

The environment-as-package model is winning. The [Prime Intellect Environments Hub](#) is creating a shared ecosystem around RL environments, in the same way PyPI and HuggingFace created ecosystems around code and model weights. Environments published once, consumed by any trainer. This is a significant infrastructure shift.

The model isn't the only variable. The training ground shapes what the model can become. The task distribution, the harness, the verifier, the state management, the topology — these are the decisions that separate agents that work from agents that demo.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {The Training Grounds: A Taxonomy of RL Environments for},
  year = {2026},
  month = {03},
  day = {21},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/03/21/rl-environm
}
```

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · [SOURCE](#)

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's *Staff engineer archetypes*. He argues that the "staff engineer" title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy

gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the “staff engineer archetypes” and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist¹. However, I think it’s bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let’s take the “team lead” archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who’s good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully ship. If you want to fill this role, shouldn’t you start trying to do these things? No! You don’t become a technical leader by trying really hard to be a technical leader, much like you don’t become a writer by trying really hard “to be a writer”. You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *Ratchet effects determine engineer reputation at large companies*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you’re familiar enough with the system and you’ve built enough trust to take on slightly larger pieces. At each stage, if you do good work - “good work” here means “deliver shareholder value” - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you’re going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you’ll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!
- You likely won’t have a good enough relationship with senior management to know what their real priorities are.
- If you’re not yet trusted to execute, you may get assigned “minders” (often current staff engineers) who will ghost-lead the project through you².
- You’ll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can’t design software you don’t work on. The “solver” and “right hand” archetypes both rely on having an enormous amount of trust and influence. You can’t aim for those archetypes directly, because trust and

influence accumulate over time. In fact, the idea of “aiming for” a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don’t get the support needed to succeed) that’s their manager’s problem, not theirs³. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can’t be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn’t possible to conjure useful work from thin air. That’s actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don’t realize is that all senior management and executive leadership roles are unfair too, in the same way. That’s just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather⁴. “Staff engineer” is the first engineering role where you are held largely responsible for outcomes you don’t control.

Developing a “staff engineer mindset” thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself “is this useful to the company” (and answering correctly).
2. Lose the habit of worrying about if you’re being treated “fairly”. Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won’t look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You’ll also look like someone who’s paying a lot of attention to what their manager’s actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you’ll eventually find yourself in one of the staff engineer archetypes. However, it probably won’t be the one you’re “aiming for”. The whole point of being a staff engineer is that you’re willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role⁵. At the time, the “staff engineer” role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they’ve built a deep relationship of trust with their organization’s leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don’t have the formal “staff” title at all, while others have “staff” as a fairly early rung on the ladder and a panoply of “senior staff”, “senior principal staff”, and so on roles above it. Like all “staff engineer” discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

↩

2. Impressing your VP’s trusted lieutenants can actually be a good way to build trust in the medium-term, but you’d better hope you’ve built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

↩

3. In theory, at least. In practice it’s always better to be useful (again, in the sense of “delivering shareholder value”).

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are [here](#) and [here](#).

The future of enterprise software

AARON LEVIE · 28 JAN 2026 · [SOURCE](#)

One of the biggest topics right now in the tech world is what does the future of software look like when AI agents are doing a substantial amount of work in the enterprise. Does software get compressed and simply become a database that agents leverage? Do we use AI agents to code all our own personalized enterprise software in the future? Do startups or incumbents win this new battle? How big are software markets in a world of agents using these tools? And much much more.

It's impossible to try and tackle all of these questions sufficiently in one sitting, but I figured I'd lay out a bit of a roadmap for what enterprise software looks like in the future.

Why companies buy software

Firstly, we should all get on the same page about why companies use and buy enterprise software in the first place. You can think of enterprise software as a codification of a company's processes. These are the parts of your company that you *don't* want to change spontaneously, because they provide the structure for your workflows and operations to keep your business running, or allow you to do the other work that really matters.

A large car company decides to procure an ERP system because it's the backbone for complex global operations where failures could mean far more than the cost of the system. It's responsible for keeping track of tens or hundreds of billions in revenue, and the company wants to ensure that every single transaction that they make goes through the exact same way every single time, with five 9s of uptime and reliability, and with an output that they can get accepted and approved by the auditors. Jobs are on the line, all the way up to the CEO, based on how well this system performs.

And the reason that companies choose to use software vendors that specialize in certain domains instead of just building this technology themselves can best be understood by the concept of "core" vs. "context," popularized by Geoffrey Moore. The core stuff is what makes your company unique (like your core product, your talent, your culture, and so on), and the context is

absolutely necessary, but largely non-differentiating between firms (think all of your systems of record like payroll, IT ticket responses, ERP systems, content management platforms, and so on).

For software that handles your company's context activities, your customer will rarely notice when these things work great, but they'll certainly notice the downstream impacts when they don't. This is why you "rent" these services from software providers that do this as their day job for thousands or millions of other companies. And incidentally, much of this software helps define these underlying workflows in your organization so you don't have to -- so each company doesn't have to reinvent the wheel each on how to handle HR, IT ticketing, and so on.

Deterministic vs. non-deterministic systems

Ok, but if an enterprise is filled with 100X more AI Agents in the future, won't that eventually consume all the use-cases that software did previously?

If you take the analogy of the industrial world, software is like the machinery in a manufacturing plant; and agents are like the people that operate that machinery. Just like machinery in a plant, in deterministic software, you want things to happen the same way, every time: you want data that goes into your ERP system to be calculated based on the business rules you've setup; you want security permissions you've set for accessing critical files to behave the same way every time, and not leak information between users; and you don't want an HR system probabilistically generating new reporting structures for employees on every load.

Conversely, you want *non-deterministic* systems (in this case AI agents) for the things that people have always tended to do in these systems. An agent filling out notes from a user's client meeting in a CRM system, generating a sales presentation for a new customer, answering a customer support inquiry with the best products to leverage, doing research or providing analysis on a set of enterprise documents and data for an important decision, and so on. As a result, the non-deterministic and deterministic elements of software end up working in a complementary fashion.

In contrast to some of the public discussion, I'd argue that in a world of 100X more AI agents than people in an enterprise, the value of the systems of record and tools agents will use will go up, not down. Because in this new world, software provides the guardrails on which agents can operate successfully within an enterprise, and gives them the underlying tools to use to be more productive themselves and work alongside people.

For instance, if you ask an AI agent to generate 10X the number of outbound email and marketing messages, where will all those agents find the data to work with, and where will the leads end up going to have a human sales rep act on them? Almost certainly some form of a CRM system. Or when an AI Agent processes reviews invoices and transactions, where will they enter this data when they're done to kick off a supply chain workflow? Again, an existing or reinvented ERP system. As long as we have humans and agents interacting with one another, there will need to be deterministic software that bridges these interactions seamlessly.

And as a result of all of these new agentic users within our software systems, the size of enterprise software markets are going to increase massively.

Unconstrained software markets

Traditionally, software companies have been stuck within the constraints of existing IT budgets, which tend to tap out around 3-7% of a company's revenue. This creates an inherent upper limit for what the budget can be for software, which translates into the total addressable market of various technology categories. Now, with AI Agents, the software is actually bringing along the work with the software, which means the budget software players are going after is the total spend that goes into doing that work in the company, not just the tech to enable it. This inevitably leads to a substantial increase in TAM for most software categories whose markets were artificially held back in size previously.

For instance, a contract management vendor just a few years ago was limited by the number of attorneys within an organization that needed to manage and review contracts. However, in a world of AI agents, that cap no longer exists, where you might have agents running continuously processing and reviewing contracts and providing extend legal services for any company. The size of the legal services market in the US is somewhere around \$400B, nearly 50-100X larger than the corresponding software categories in legal space. You can easily see how AI agents making the legal industry meaningfully more productive could capture multiple percentage points of that \$400B in spend, thus multiplying the size of the legal software market overnight.

The same analogy holds for almost any software category that was artificially constrained by a customer's headcount, or limited software spend, previously. We're already seeing this in spaces like coding, where the AI agents in coding are generating orders of magnitude more revenue for the new coding startups than the prior IDE categories generated. And we can expect this same dynamic for most other areas of knowledge work over time, from healthcare and financial services to legal and marketing. Tools that bring along work will generate far more revenue than the prior era of tools used to.

The opportunity ahead for software

Now, the big open question is who wins in this technology wave. If we have learned anything from Clayton Christensen's Innovator's Dilemma framework, we know that incumbents tend to lose in markets when there's a new tech or business model innovation that is unattractive to them because it represents less revenue or profitability. Conversely, they tend to hold their own when the new business model or technology is supportive of their existing business, or perhaps even enhances it. Ultimately, in AI, we'll likely see examples of both outcomes.

Some existing software categories, like customer support or coding, are changing so rapidly that many incumbents will be unable to change their business models or technology stacks fast enough to respond to the growing demands of the new market, even if getting to the other side would be economically attractive. Similarly, as Jaya Gupta and Ashu Garg outlined in their piece on context graphs, there are many new categories of agent-native software that will need to be created for the first time because no existing vendor logically can capture the way the agent needs to do its work. This will create tremendous opportunities for new startups to build AI agents within these categories, and we're seeing new ones crop up every day.

Conversely, there are a number of existing SaaS players are in a natural position to power agents in their relevant work categories, especially those that have the natural context necessary for AI agents to be successful in automating work. Unsurprisingly, I'd expect there to be a correlation between the strength of a software company's moat and the amount of data they house, how complex the underlying workflows are, the number of connections there are with other systems, how intertwined a human-in-the-loop element is to the underlying agentic workflow, and how naturally AI agents can be plugged into these workflows (either as native users or via API in external agentic products).

Of course, this will mean that the business model of software (new upstarts and incumbents) will have to evolve over time. Today, the vast majority of SaaS products charge on a per-seat basis, which generally corresponds to most of the usage that the software sees today by its end users. But in a world where AI agents do most of the interaction and work on software, enterprise systems will have to evolve to support more of a consumption and usage-based model over time. AI agents don't cleanly fit as seats on software, because any given AI agent can do a varied amount of work within a system (e.g. you could have 1 agent doing a billion things or a billion agents doing one thing).

Inevitably, we'll see a mix of both outcomes. Users (or people) will continue to be seats within software, and AI agents that represent extended usage will be monetized through consumption. We're already seeing this play out in AI-native coding products and other platforms where there's an end-user seat component mixed with a consumption model on top. Obviously depending on the human vs. agent value proposition mix, the revenue stream will slide between these two ends of the continuum.

Ultimately, there's still plenty that needs to play out before we fully see what the future holds for enterprise software. But it will certainly be the most dynamic and exciting period we've ever seen in software history.